

Contents

□ Monotonic Queue / Deque — Complete Interview Handbook	1
Table of Contents	1
SECTION 1 — Pattern Overview	2
SECTION 2 — Recognition Guide	3
SECTION 3 — Decision Framework	4
SECTION 4 — Why This Pattern Works	4
SECTION 5 — Algorithm Framework	5
SECTION 6 — Time & Space Complexity	5
SECTION 7 — Edge Cases	6
SECTION 8 — Pros & Cons	6
SECTION 9 — Real World Applications	7
SECTION 10 — Interview Strategy	7
SECTION 11 — Pattern Variations	7
SECTION 12 — Comparison With Other Patterns	8
SECTION 13 — Problem Classification	8
SECTION 14 — 30 Interview Problems	9
SECTION 15 — Common Mistakes	12
SECTION 16 — Optimization Techniques	12
SECTION 17 — Multiple Language Templates	13
SECTION 18 — Dry Runs	15
SECTION 19 — Advanced Concepts	15
SECTION 20 — Senior Engineer Insights	15
SECTION 21 — Cheat Sheet	16
SECTION 22 — Revision Notes (5-Minute Review)	16
SECTION 23 — Practice Roadmap	16
SECTION 24 — Memory Tricks	17
SECTION 25 — Final Summary	17

□ **Monotonic Queue / Deque — Complete Interview Handbook**

Pattern #11 of 28 | DSA Pattern Mastery Series Audience: Senior/Staff SWE interviews (Google, Amazon, Meta, Microsoft, Uber, Airbnb, Atlassian, Grab, Careem, Noon, Talabat, Dubai/Saudi & India Tier-1/Tier-2 product companies) **Companion:** Pairs with Striver's A2Z DSA Course — Stack & Queue section

Table of Contents

1. Pattern Overview · 2. Recognition Guide · 3. Decision Framework · 4. Why This Pattern Works · 5. Algorithm Framework · 6. Time & Space Complexity · 7. Edge Cases · 8. Pros & Cons · 9. Real World Applications · 10. Interview Strategy · 11. Pattern Variations · 12. Comparison With Other Patterns · 13. Problem Classification · 14. 30 Interview Problems · 15. Common Mistakes · 16. Optimization Techniques · 17. Multiple Language Templates · 18. Dry Runs · 19. Advanced Concepts · 20. Senior Engineer Insights · 21. Cheat Sheet · 22. Revision Notes · 23. Practice Roadmap · 24. Memory Tricks · 25. Final Summary

SECTION 1 — Pattern Overview

1.1 What Is This Pattern?

A Monotonic Deque (double-ended queue) maintains its elements in monotonic order (increasing or decreasing) while supporting **eviction from both the front and back**. This makes it the tool of choice for finding the **minimum or maximum within a sliding window** in $O(n)$ total, since — unlike a Monotonic Stack — elements must be removed from the front once they slide “out of window,” in addition to being removed from the back when dominated.

1.2 Why Was This Pattern Invented?

Naively recomputing the min/max of every sliding window costs $O(n \cdot k)$ (k = window size) or $O(n \log k)$ with a heap. The insight: **within a window, any element that is smaller (for a max-query) than a later element it precedes can never be the answer for any future window that contains both** — it is permanently dominated the instant a “better” element appears after it, *as long as that better element stays within the window*. This lets you maintain only the “still-possibly-relevant” candidates in a deque, evicting dominated ones from the back and out-of-window ones from the front, in $O(n)$ total.

1.3 Real Intuition Behind The Pattern

Imagine a **queue of people waiting to be “tallest person in the last 5 minutes.”** As new people arrive, anyone shorter than the newest arrival who’s still in the queue can be dismissed immediately — they can never win “tallest in the last 5 minutes” once someone taller and more recent exists. Separately, anyone who’s been waiting more than 5 minutes ages out from the front, regardless of height.

1.4 Mental Model

The deque holds indices in a monotonic order (e.g., decreasing values, front to back, for a sliding-window-maximum problem) representing “candidates still eligible to be the window’s max.” Two independent removal conditions operate: (1) pop from the **back** while the new element dominates it (maintains monotonicity), and (2) pop from the **front** while the front index has slid out of the current window (maintains window relevance).

1.5 Visual Explanation

`arr = [1, 3, -1, -3, 5, 3, 6, 7], k = 3` (sliding window maximum)

```
i=0(1): deque=[] → push 0 → deque=[0]
i=1(3): 3 > arr[0]=1 → pop 0 → deque=[] → push 1 → deque=[1]
i=2(-1): -1 < arr[1]=3 → push 2 → deque=[1,2]
        window [0,2] complete → max = arr[deque.front]=arr[1]=3
i=3(-3): -3 < arr[2]=-1 → push 3 → deque=[1,2,3]
        front(1) still in window [1,3]? yes → max = arr[1]=3
i=4(5): 5 > arr[3]=-3 → pop 3; 5 > arr[2]=-1 → pop 2; 5 > arr[1]=3 → pop 1 → deque=[] → push 4 → deque=[4]
        window [2,4] → max = arr[4]=5
i=5(3): 3 < arr[4]=5 → push 5 → deque=[4,5]
        window [3,5] → front(4) in window → max=arr[4]=5
...continues similarly
```

1.6 Simple Analogy

Monotonic Deque is like a **nightclub bouncer’s VIP line that both ages people out (they’ve been waiting too long) and dismisses people who are clearly less important than someone**

who just walked in — the line always stays sorted by “importance,” and the person at the very front is always the current best candidate, as long as they haven’t been waiting too long.

1.7 When Should I Immediately Think About Using This Pattern?

- “Sliding window **maximum/minimum**” over a fixed-size window.
- Problems combining **Sliding Window** (Pattern #2) with a need for the window’s running extremum, not just its sum/count.
- “Shortest subarray with sum at least K” (uses a monotonic deque over prefix sums).
- Any $O(n \cdot k)$ or $O(n \log k)$ brute-force/heap solution for windowed extremes that could be optimized to $O(n)$.

SECTION 2 — Recognition Guide

2.1 Keywords

Keyword	Signal
“sliding window maximum/minimum”	Direct signal
“maximum in every window of size k”	Direct signal
“shortest subarray with sum at least K”	Monotonic deque over prefix sums
“constrained subsequence sum”	Monotonic deque over DP values within a window
“jump game” style with window constraint on reachable max	Monotonic deque for windowed DP optimization

2.2 Hidden Hints

A problem that looks like Sliding Window (Pattern #2) but explicitly asks for the window’s **min/max**, not sum/count/distinct-elements — this specific need is the tell that a plain window with a simple running variable won’t suffice; you need the full deque.

2.3 Interview Clues

Interviewer asks “can you avoid recomputing the max for every window?” after you propose an $O(n \cdot k)$ brute force, or “can you beat $O(n \log k)$ ” after you propose a heap-based solution.

2.4 Common Trick Words

“Maximum/minimum of all subarrays of size k,” “at most a distance apart” combined with a min/max requirement — these phrasings point to Monotonic Deque specifically, not a simple Sliding Window.

2.5 What Interviewers Expect

Correct dual-eviction logic (back for dominance, front for window expiry), correct choice of increasing vs decreasing deque, and the $O(n)$ amortized argument (each element pushed once, popped at most once from either end).

2.6 When NOT To Use This Pattern

- You only need the window’s **sum or count**, not min/max — plain Sliding Window with a running variable suffices, no deque needed.

- You need the **k-th smallest/largest** (not just the extremum) within the window — that needs a balanced structure like two heaps or an order-statistics tree, not a monotonic deque.
 - The “window” isn’t contiguous/fixed-size in a simple sense — e.g., arbitrary subsequences — this pattern doesn’t apply.
-

SECTION 3 — Decision Framework

Do you need the MIN or MAX within a sliding/moving window?

Yes

Is the window FIXED SIZE (or bounded by a simple index/value constraint)?

Yes → USE MONOTONIC DEQUE ($O(n)$ total)

No

Do you only need SUM/COUNT/DISTINCT within the window (not min/max)?

Yes → USE PLAIN SLIDING WINDOW (Pattern #2) with a running variable - no deque needed

No

Do you need the K-TH SMALLEST/LARGEST (not just min/max) within the window?

Yes → USE TWO HEAPS / ORDER-STATISTICS STRUCTURE instead (more complex, $O(n \log k)$)

Why: Monotonic Deque’s specific value is $O(n)$ windowed min/max — a narrower need than general windowed aggregation (handled by simple Sliding Window) or general windowed order statistics (needing heaps/balanced trees). Misidentifying which of these three is needed leads to either overcomplicating or under-solving the problem.

SECTION 4 — Why This Pattern Works

Mathematical/Logical: Each index is pushed onto the deque exactly once (when first encountered) and popped at most once (either from the back due to dominance, or from the front due to window expiry) — total operations bounded by $3n = O(n)$ (n pushes + at most n back-pops + at most n front-pops).

Intuitive: Two independent, simple invariants — “the deque is monotonic” and “the deque only contains in-window indices” — together guarantee the front of the deque is always the current window’s extremum, without ever needing to re-scan the window’s contents.

Correctness Proof: *Invariant:* at any point after processing index i , the deque contains, in monotonically decreasing order (for a max-query) from front to back, exactly those indices in the current window $[i-k+1, i]$ that are **not dominated** by any later index in the same window. *Base case:* trivially true for the first window formed. *Inductive step:* when a new index $i+1$ arrives, popping back-elements smaller than $arr[i+1]$ removes exactly the now-dominated indices (since $arr[i+1]$ is both later and bigger, dominating them for all remaining windows they’d share); popping front-elements outside $[i+1-k+1, i+1]$ removes exactly the expired indices. Both operations preserve the invariant. *Termination:* the front of the deque after each window update is, by the invariant, the maximum of the current window. **QED.**

SECTION 5 — Algorithm Framework

5.1 Step-by-Step Framework (Sliding Window Maximum)

1. Initialize an empty deque (storing indices).
2. For each index i from 0 to $n-1$:
 - a. While the deque is non-empty and $\text{arr}[\text{deque.back}] \leq \text{arr}[i]$, pop from the back (dominance eviction).
 - b. Push i to the back.
 - c. While $\text{deque.front} \leq i - k$ (out of window), pop from the front (expiry eviction).
 - d. Once $i \geq k - 1$, record $\text{arr}[\text{deque.front}]$ as the current window's maximum.

5.2 General Template

```
function slidingWindowMaximum(arr, k):
    deque = []                # stores indices, decreasing values front to back
    result = []
    for i in range(0, length(arr)):
        while deque is not empty and arr[deque.back()] <= arr[i]:
            deque.popBack()
        deque.pushBack(i)

        if deque.front() <= i - k:
            deque.popFront()

        if i >= k - 1:
            result.append(arr[deque.front()])
    return result
```

5.3 Interview Thinking Process

1. "This needs the window's max, not just sum — a plain sliding window with a running sum won't work; I need a Monotonic Deque."
2. "I'll evict from the back whenever a new, later element is at least as big (it dominates everything smaller before it for all future shared windows)."
3. "I'll evict from the front whenever the front index has slid outside the current window."
4. "The front of the deque is always the current window's max, once the window is fully formed ($i \geq k-1$)."
5. "Total work is $O(n)$ amortized — each index is pushed once and popped at most once, from either end."

SECTION 6 — Time & Space Complexity

Case	Time	Space	Why
Worst Case	$O(n)$	$O(k)$ (deque size bounded by window size)	Each index pushed once, popped at most once total
Average Case	$O(n)$	$O(k)$	Same regardless of data distribution
Best Case	$O(n)$ (still must scan once)	$O(1)$ to $O(k)$	Even trivial windows require a full scan

Case	Time	Space	Why
Amortized	$O(n)$ despite the nested-looking while-loops	$O(k)$	Total pushes + pops (both ends) bounded by $3n$

Comparison: Naive recomputation is $O(n \cdot k)$; heap-based approach is $O(n \log k)$; Monotonic Deque achieves $O(n)$ — strictly better than both.

SECTION 7 — Edge Cases

Edge Case	Example	Handling
$k = 1$	Every element is its own window	Result equals the input array itself
$k = n$ (whole array is one window)	Single output value	Deque correctly reduces to the global max
Empty array	<code>[]</code>	Return empty result immediately
All identical elements	<code>[5,5,5,5]</code> , $k=2$	Using $\leq$ for back-eviction correctly keeps only the most recent duplicate, still giving the correct max
Strictly increasing array	<code>[1,2,3,4,5]</code> , $k=3$	Every new element evicts the entire back of the deque — deque size never exceeds 1 in practice for this case
Strictly decreasing array	<code>[5,4,3,2,1]</code> , $k=3$	No back-evictions ever occur; front-evictions handle window expiry as the array progresses
Window larger than array	$k > n$	No valid window exists — return empty or handle per problem spec

Common mistakes: using $<$ instead of $\leq$ for back-eviction with duplicate values, causing stale duplicate indices to linger unnecessarily (still correct in outcome but wasteful, and can cause bugs in variants tracking exact index identity); forgetting the front-eviction check entirely, causing expired indices to incorrectly remain “in window.”

SECTION 8 — Pros & Cons

Advantages: $O(n)$ time for sliding-window min/max — strictly better than $O(n \cdot k)$ brute force or $O(n \log k)$ heap-based approaches; $O(k)$ space, bounded by window size. **Disadvantages:** Requires careful dual-eviction logic (back for dominance, front for expiry) — a common source of subtle bugs; conceptually more involved than a plain Monotonic Stack. **Trade-offs:** Monotonic Deque ($O(n)$ time, $O(k)$ space) vs. Heap-based ($O(n \log k)$ time, $O(k)$ space, simpler to reason about but slower) — prefer the deque for the tightest possible complexity, the heap when code simplicity is prioritized over asymptotic optimality. **Limitations:** Only answers min/max, not k -th order statistics; only applies to contiguous, typically fixed-size windows. **Inefficient when:** N/A for its exact use case — $O(n)$ is optimal, since every element must be examined at least once.

SECTION 9 — Real World Applications

Domain	Application
Google/Meta	Real-time analytics dashboards computing rolling max/min metrics (e.g., peak CPU usage in the last 5-minute window)
Amazon	Real-time price monitoring — rolling maximum/minimum price over a trailing time window for dynamic pricing alerts
Netflix	Adaptive streaming — rolling maximum observed bandwidth over a trailing window to inform bitrate decisions
Finance/Trading	Rolling maximum/minimum stock price over a trailing window (technical indicators like rolling highs/lows)
Networking	Sliding window congestion control tracking maximum observed round-trip-time over a trailing packet window
Image Processing	Sliding window max/min filters (morphological dilation/erosion) over pixel neighborhoods
Signal Processing	Rolling peak detection in sensor data streams (e.g., IoT temperature/vibration monitoring)
Operating Systems	Rolling maximum resource usage tracking for adaptive throttling/scheduling decisions

SECTION 10 — Interview Strategy

How seniors answer: They immediately distinguish this from plain Sliding Window (“I need the running max, not sum — a simple running variable won’t work since removing an element from the window doesn’t tell me the new max in $O(1)$ ”), then describe the dual-eviction deque mechanism and its $O(n)$ amortized proof before coding.

How juniors answer: They often propose recomputing the max via a nested loop ($O(n \cdot k)$) or reach for a heap without recognizing that a heap can’t efficiently remove the *specific* element that slides out of the window (requires lazy deletion), missing the cleaner $O(n)$ deque solution.

Typical follow-ups: “Why can’t you just track a running max like you would for sum?” (Because removing an element from the window may invalidate the running max, requiring a full re-scan unless you maintain more structure — hence the deque.) “How would you extend this to the k -th largest in the window?” (Discuss two heaps / balanced BST instead.)

Optimization questions: “Can you avoid the $O(k)$ space?” (Generally no, the deque size is fundamentally bounded by window contents needed for correctness.)

SECTION 11 — Pattern Variations

Variation	Description	Example
Sliding Window Maximum	Decreasing deque, track max	Sliding Window Maximum
Sliding Window Minimum	Increasing deque, track min	Sliding Window Minimum (variant)
Shortest Subarray with Sum at Least K	Monotonic deque over prefix sums (handles negative numbers)	Shortest Subarray with Sum at Least K
Constrained Subsequence Sum	Monotonic deque over DP array within a window	Constrained Subsequence Sum
Jump Game VI	Monotonic deque optimizing windowed DP transitions	Jump Game VI
Sliding Window Median (contrast)	Needs two heaps, not a monotonic deque	Sliding Window Median

SECTION 12 — Comparison With Other Patterns

Pattern	Difference	When To Prefer
Monotonic Stack	Single-end eviction only, no window-expiry concept	Nearest greater/smaller in one direction, not windowed extremes
Plain Sliding Window	Tracks running sum/count with $O(1)$ incremental update	Window's aggregate is sum/count/distinct, not min/max
Heap (Two Heaps / Priority Queue)	Handles k-th order statistics, but $O(\log k)$ per operation and awkward removal of arbitrary (expired) elements	Need median or k-th smallest/largest within the window
Prefix Sum + Monotonic Deque (combined)	Used together for "shortest subarray with sum at least K" — deque over prefix sum indices, not raw values	Range-sum-with-negatives problems needing windowed monotonicity

Comparison Table

Aspect	Monotonic Deque	Heap-Based Window	Plain Sliding Window
Time	$O(n)$	$O(n \log k)$	$O(n)$
Handles min/max	Yes	Yes	No (only sum/count)
Handles k-th order stat	No	With two heaps, yes	No
Space	$O(k)$	$O(k)$	$O(1)$ to $O(k)$

SECTION 13 — Problem Classification

Difficulty	Characteristics	Examples
Easy	N/A (this pattern rarely appears at Easy difficulty)	—
Medium	Direct sliding window max/min	Sliding Window Maximum (framed as Medium in some judges)
Hard	Combined with prefix sums or DP for advanced constraints	Shortest Subarray with Sum at Least K, Constrained Subsequence Sum, Jump Game VI
Very Hard	Multi-constraint combinations, 2D extensions	Sliding window extremes over 2D grids, advanced DP-optimization variants

SECTION 14 — 30 Interview Problems

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
1	Sliding Window Maximum	Hard	Amazon, Google, Meta, Microsoft	Direct decreasing monotonic deque	Foundational mechanics
2	Sliding Window Minimum (variant)	Medium	Google	Increasing monotonic deque	Mirror-image mastery
3	Shortest Subarray with Sum at Least K	Hard	Google, Amazon, Uber	Monotonic deque over prefix sums	Advanced prefix sum + deque combination
4	Constrained Subsequence Sum	Hard	Google, Amazon	Monotonic deque optimizing DP transitions	DP + deque optimization
5	Jump Game VI	Medium	Google, Amazon	Monotonic deque for windowed DP max	DP + deque optimization
6	Sliding Window Median	Hard	Google, Amazon	Contrast: needs two heaps, not deque	Pattern-boundary awareness
7	Longest Continuous Subarray With Absolute Diff $\leq$ Limit	Medium	Google, Meta	Dual monotonic deque (min and max simultaneously)	Dual-deque combination

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
8	Maximum of Minimum Values in All Subarrays	Medium	Google	Related monotonic stack/deque hybrid reasoning	Cross-pattern combination
9	Sum of Subarray Minimums (contrast)	Medium	Amazon, Google	Contrast: Monotonic Stack, not Deque (no window expiry)	Pattern-boundary awareness
10	Maximum Number of Robots Within Budget	Hard	Google (advanced)	Monotonic deque combined with sliding window sum constraint	Advanced dual-constraint window
11	Shortest Subarray to be Removed to Make Array Sorted (contrast)	Medium	Google, Amazon	Contrast: Two Pointers, not Deque	Pattern-boundary awareness
12	Max Value of Equation	Hard	Google	Monotonic deque for windowed max with linear constraint	Advanced constraint-based deque
13	Find the Most Competitive Subsequence	Medium	Google, Amazon	Monotonic stack/deque hybrid for subsequence construction	Constructive deque application
14	Minimum Number of K Consecutive Bit Flips	Hard	Google	Difference array + deque-style window tracking	Advanced hybrid technique
15	Maximum Subarray Min-Product	Medium	Amazon, Google	Combines prefix sum with monotonic stack/deque	Cross-pattern combination
16	Sliding Puzzle (contrast, BFS problem)	Hard	Google	Contrast: BFS on states, not monotonic deque	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
17	Continuous Subarrays (Codeforces/interview variant)	Medium	Google (custom)	Dual monotonic deque for range constraint	Dual-deque application
18	Maximum Score of a Good Subarray	Hard	Google	Two Pointers + monotonic reasoning hybrid	Cross-pattern combination
19	Number of Visible People in a Queue (contrast)	Hard	Google, Amazon	Contrast: Monotonic Stack (one direction), not Deque	Pattern-boundary awareness
20	Maximum Sliding Window Sum with Constraint (custom/interview variant)	Medium	Amazon (conceptual)	Combines Sliding Window sum with Deque-tracked max	Hybrid combination
21	Shortest Subarray With OR at Least K (custom variant)	Hard	Google (advanced)	Monotonic deque adaptation for bitwise constraints	Advanced bitwise-window hybrid
22	Longest Subarray With Maximum Bitwise AND (contrast)	Medium	Google	Contrast: simple scan, not deque needed	Pattern-boundary awareness
23	Minimum Window Substring (contrast)	Hard	Meta, Amazon	Contrast: Sliding Window + Hashing, not Deque	Pattern-boundary awareness
24	Trapping Rain Water II (contrast, uses heap)	Hard	Google, Amazon	Contrast: heap-based, not deque	Pattern-boundary awareness
25	Frequency of the Most Frequent Element (contrast)	Medium	Google, Amazon	Contrast: Sliding Window + sort, not deque	Pattern-boundary awareness
26	Maximum Points You Can Obtain from Cards (contrast)	Medium	Amazon	Contrast: prefix sum + two pointers, not deque	Pattern-boundary awareness

#	Problem	Difficulty	Companies	Why Pattern Applies	Learning Objective
27	Grumpy Bookstore Owner (contrast)	Medium	Amazon	Contrast: fixed window sum, not deque	Pattern-boundary awareness
28	Sliding Window Maximum II (custom harder variant)	Hard	Google (conceptual)	Extension with multiple simultaneous windows	Advanced multi-window deque
29	Maximum Average Pass Ratio (contrast, uses heap)	Medium	Google	Contrast: greedy + heap, not deque	Pattern-boundary awareness
30	Design a Rolling Max/Min Data Structure (system design-adjacent)	Custom/Advanced	Google, Amazon	Direct application of Monotonic Deque as a production data structure	Applied system design

SECTION 15 — Common Mistakes

1. Forgetting the front-eviction (window-expiry) check entirely, treating this like a Monotonic Stack — causes stale, out-of-window indices to be incorrectly considered. *Fix:* always check and evict from the front based on the window boundary before reading the result.
2. Using strict $<$ instead of $\leq$ (or vice versa) for back-eviction when duplicates matter — usually harmless for the max value itself but can cause bugs in variants that care about exact index identity. *Fix:* be deliberate and test with duplicate-containing input.
3. Reading the result (`arr[deque.front]`) before the window is fully formed ($i < k - 1$), producing premature/incorrect partial-window results. *Fix:* always guard the result-recording step with $i \geq k - 1$.
4. Confusing this pattern with Monotonic Stack and omitting the front-eviction logic, especially when adapting a “next greater element” solution. *Fix:* explicitly recognize the two separate eviction conditions before coding.
5. Forgetting to store indices (not values) — needed to correctly check window-expiry (`front index  $\leq i - k$`). *Fix:* always store indices for windowed variants.

Why people fail: the pattern looks like a simple hybrid of Sliding Window and Monotonic Stack, but combining both correctly (two separate, simultaneously-maintained invariants) is more intricate than either alone — candidates who understand each half individually sometimes fail to correctly interleave both eviction conditions under interview time pressure.

SECTION 16 — Optimization Techniques

- **Time:** Already optimal at $O(n)$ — no further asymptotic improvement possible for this exact problem shape.

- **Space:** Bounded naturally by $O(k)$ since the deque never holds more than the current window's worth of “undominated” candidates; no further optimization typically needed.
 - **Readability:** Clearly separate the “back eviction” (dominance) and “front eviction” (expiry) logic into distinct, well-commented blocks rather than merging them into one dense conditional.
 - **Interview performance:** Explicitly contrast this with plain Sliding Window and Monotonic Stack before coding — demonstrating you understand exactly which of the three techniques the problem needs is a strong signal.
-

SECTION 17 — Multiple Language Templates

Java

```
public int[] maxSlidingWindow(int[] nums, int k) {
    Deque<Integer> deque = new ArrayDeque<>();
    int[] result = new int[nums.length - k + 1];
    for (int i = 0; i < nums.length; i++) {
        while (!deque.isEmpty() && nums[deque.peekLast()] <= nums[i]) deque.pollLast();
        deque.offerLast(i);
        if (deque.peekFirst() <= i - k) deque.pollFirst();
        if (i >= k - 1) result[i - k + 1] = nums[deque.peekFirst()];
    }
    return result;
}
```

JavaScript

```
function maxSlidingWindow(nums, k) {
    const deque = [];
    const result = [];
    for (let i = 0; i < nums.length; i++) {
        while (deque.length && nums[deque[deque.length-1]] <= nums[i]) deque.pop();
        deque.push(i);
        if (deque[0] <= i - k) deque.shift();
        if (i >= k - 1) result.push(nums[deque[0]]);
    }
    return result;
}
```

PHP

```
function maxSlidingWindow(array $nums, int $k): array {
    $deque = [];
    $result = [];
    for ($i = 0; $i < count($nums); $i++) {
        while (!empty($deque) && $nums[end($deque)] <= $nums[$i]) array_pop($deque);
        $deque[] = $i;
        if ($deque[0] <= $i - $k) array_shift($deque);
        if ($i >= $k - 1) $result[] = $nums[$deque[0]];
    }
    return $result;
}
```

Python

```
from collections import deque as Deque
def max_sliding_window(nums, k):
    dq = Deque()
    result = []
    for i, num in enumerate(nums):
        while dq and nums[dq[-1]] <= num:
            dq.pop()
        dq.append(i)
        if dq[0] <= i - k:
            dq.popleft()
        if i >= k - 1:
            result.append(nums[dq[0]])
    return result
```

Go

```
func maxSlidingWindow(nums []int, k int) []int {
    var deque []int
    var result []int
    for i, num := range nums {
        for len(deque) > 0 && nums[deque[len(deque)-1]] <= num {
            deque = deque[:len(deque)-1]
        }
        deque = append(deque, i)
        if deque[0] <= i-k {
            deque = deque[1:]
        }
        if i >= k-1 {
            result = append(result, nums[deque[0]])
        }
    }
    return result
}
```

C++

```
vector<int> maxSlidingWindow(vector<int>& nums, int k) {
    deque<int> dq;
    vector<int> result;
    for (int i = 0; i < (int)nums.size(); i++) {
        while (!dq.empty() && nums[dq.back()] <= nums[i]) dq.pop_back();
        dq.push_back(i);
        if (dq.front() <= i - k) dq.pop_front();
        if (i >= k - 1) result.push_back(nums[dq.front()]);
    }
    return result;
}
```

SECTION 18 — Dry Runs

Small Input

nums = [1,3,-1,-3,5,3,6,7], k = 3 (matches \$1.5 visual, extended to completion)

```
i=0(1): deque=[0]
i=1(3): pop 0 (1<=3) → deque=[1]
i=2(-1): deque=[1,2] → window complete → result=[3]
i=3(-3): deque=[1,2,3] → front(1) in [1,3] → result=[3,3]
i=4(5): pop 3,2,1 (all <=5) → deque=[4] → result=[3,3,5]
i=5(3): deque=[4,5] → front(4) in [3,5] → result=[3,3,5,5]
i=6(6): pop 5,4 (both <=6) → deque=[6] → result=[3,3,5,5,6]
i=7(7): pop 6 (6<=7) → deque=[7] → result=[3,3,5,5,6,7]
Final: [3,3,5,5,6,7]
```

Large Input (Conceptual)

For 10^6 elements with $k=1000$, total deque operations remain bounded by $\sim 3 \times 10^6$ (each index pushed once, popped at most once from either end), regardless of k 's size — confirming true $O(n)$, independent of window size, unlike the $O(n \cdot k)$ brute force.

Corner Case

nums = [5], k = 1: i=0: deque=[0], window complete immediately ($i \geq k-1 = 0$) → result=[5], correctly handling the trivial single-element window.

SECTION 19 — Advanced Concepts

- **Monotonic Deque over transformed values (prefix sums):** for “Shortest Subarray with Sum at Least K” (which allows negative numbers, breaking simple Sliding Window’s monotonicity), maintain a monotonic deque of **prefix sum indices** instead of raw array values — searching for the shortest valid subarray becomes a windowed-minimum-prefix-sum problem solvable with the same dual-eviction deque technique.
 - **DP optimization via Monotonic Deque:** in “Constrained Subsequence Sum” and “Jump Game VI,” the recurrence $dp[i] = arr[i] + \max(dp[i-k..i-1])$ naturally invites a monotonic deque to maintain the sliding-window max of dp values, turning an $O(n \cdot k)$ DP into $O(n)$ — a powerful, generalizable technique: **whenever a DP recurrence involves a windowed max/min over previous states, consider a monotonic deque optimization.**
 - **Dual monotonic deques:** problems needing both the window’s min AND max simultaneously (e.g., “Longest Continuous Subarray With Absolute Diff $\leq$ Limit”) maintain two separate deques (one increasing, one decreasing) in parallel, both evicted using the same window-expiry logic.
-

SECTION 20 — Senior Engineer Insights

Staff engineers recognize Monotonic Deque as the natural extension of Monotonic Stack once a **temporal/positional expiry constraint** (the “window”) is added on top of the “dominance” concept — the same amortized double-ended eviction reasoning generalizes to real-time analytics systems tracking rolling extremes (e.g., “max latency in the last 60 seconds” in an observability pipeline), where the front-eviction condition becomes “timestamp older than 60 seconds ago” rather than “index outside the last k positions.” Interviewers evaluate whether a candidate can identify the powerful generalization from “sliding window max” to “DP transition optimization via

windowed max," which is what separates advanced/Staff-level candidates on Hard-tier problems like Constrained Subsequence Sum.

SECTION 21 — Cheat Sheet

PATTERN: Monotonic Deque

RECOGNIZE: "sliding window maximum/minimum," fixed-size window + min/max requirement

TEMPLATE:

```
deque = [] # indices, monotonic front-to-back
for i in range(n):
    while deque and arr[deque.back] <= arr[i]: deque.popBack() # dominance eviction
    deque.pushBack(i)
    if deque.front <= i - k: deque.popFront() # expiry eviction
    if i >= k-1: result = arr[deque.front]
```

COMPLEXITY: $O(n)$ time, $O(k)$ space

KEY PROOF: each index pushed once, popped at most once from either end - amortized $O(n)$

WATCH FOR: dual eviction (dominance AND expiry), storing indices not values, guarding premature result reset

DOESN'T APPLY WHEN: need sum/count only (plain Sliding Window), need k-th order stat (use heaps)

SECTION 22 — Revision Notes (5-Minute Review)

- Monotonic Deque = Monotonic Stack + window-expiry eviction from the front.
 - Two eviction conditions: back (dominance) and front (out-of-window).
 - Front of the deque is always the current window's extremum, once the window is fully formed.
 - $O(n)$ amortized — each index pushed once, popped at most once from either end.
 - Use for sliding-window max/min; use plain Sliding Window for sum/count; use heaps for k-th order stat.
 - Generalizes to DP optimization whenever a recurrence needs a windowed max/min of prior states.
-

SECTION 23 — Practice Roadmap

Stage	Focus	Recommended LeetCode Problems
Beginner	Core dual-eviction mechanics	Sliding Window Maximum (239)
Intermediate	Prefix-sum combination	Shortest Subarray with Sum at Least K (862)
Advanced	DP transition optimization	Jump Game VI (1696), Constrained Subsequence Sum (1425)
Expert	Dual-deque, multi-constraint combinations	Longest Continuous Subarray With Absolute Diff $\leq$ Limit (1438), Max Value of Equation (1499)

SECTION 24 — Memory Tricks

- **Mnemonic:** “Dominate from the back, Expire from the front” (DEF).
 - **Visualization:** A **VIP line that both dismisses less-important people and ages out people who’ve waited too long** — the front is always the current best, valid candidate.
 - **Recognition shortcut:** “Sliding window” + “maximum/minimum” (not sum/count) → Monotonic Deque, immediately.
-

SECTION 25 — Final Summary

Monotonic Deque extends Monotonic Stack with a second, independent eviction rule — expiry from the front when an index slides out of the window — enabling $O(n)$ sliding-window maximum/minimum computation, strictly better than $O(n \cdot k)$ brute force or $O(n \log k)$ heap-based approaches. The single most important thing to remember forever: **maintain two separate invariants simultaneously — monotonicity (back eviction, dominance) and window relevance (front eviction, expiry) — and recognize that this same “windowed max/min” idea generalizes powerfully to optimizing DP recurrences that depend on a sliding window of prior states.**